

JEONME

Dokumentasi Backend

Go + Gin REST API — Modular Monolith

Dokumen	BE-DOC v1.0
Status	Kerangka awal (skeleton)
Tanggal	3 Juli 2026
Stack	Go 1.25 · Gin 1.10 · PostgreSQL 16 · Redis 7
Lokasi kode	apps/api/

1. Ringkasan Arsitektur

Backend mengikuti prinsip **modular monolith** dari Technical Design Document Bagian 1: satu binary Go/Gin dengan batas modul yang jelas (auth, page, product, dan seterusnya sesuai roadmap), sehingga bagian yang butuh skala tinggi — halaman publik & checkout — bisa diekstraksi menjadi service terpisah nanti tanpa menulis ulang semuanya.

Koneksi database memakai `pgxpool` (bukan `database/sql` biasa) untuk performa baca/tulis tinggi di endpoint seperti checkout, dengan graceful shutdown di `main.go` agar request yang sedang berjalan tidak terputus mendadak saat deploy/restart container.

2. Struktur Folder

```
apps/api/
├─ main.go                # Entry point + subcommand `migrate`
├─ go.mod / go.sum
├─ migrations/
│  ├─ 000001_init_schema.up.sql
│  └─ 000001_init_schema.down.sql
└─ internal/
    ├─ config/config.go    # Load & validasi environment variable
    ├─ database/database.go # pgxpool + redis client
    ├─ middleware/middleware.go # CORS, RequestLogger, AuthRequired (JWT)
    ├─ migrate/migrate.go  # Wrapper golang-migrate (subcommand `migrate`)
    ├─ models/models.go    # Struct Go: User, Page, Link, Product, Order
    ├─ routes/routes.go    # Pendaftaran seluruh route
    └─ handlers/
        ├─ health.go       # /api/health
        ├─ auth.go         # /api/v1/auth/{register,login}
        ├─ page.go         # /api/v1/pages/:username
        ├─ product.go      # /api/v1/dashboard/products
        └─ health_test.go  # Test integrasi (DB sungguhan)
```

3. Konfigurasi & Environment

`internal/config/config.go` memuat `.env` (via `godotenv`, opsional) lalu environment variable sistem. Variabel wajib (`DATABASE_URL` , `JWT_SECRET`) membuat aplikasi *fail-fast* saat startup jika belum diset — lebih baik gagal di awal daripada di tengah request pembayaran.

VARIABEL	WAJIB?	DEFAULT	KETERANGAN
APP_ENV	Tidak	local	production mengaktifkan <code>gin.ReleaseMode</code>
APP_PORT	Tidak	8080	Port HTTP server

DATABASE_URL	Ya	—	Format postgres://user:pass@host:5432/db?sslmode=disable
REDIS_URL	Tidak	redis://localhost:6379/0	—
JWT_SECRET	Ya	—	Random string panjang, beda per environment
XENDIT_SECRET_KEY	Tidak (untuk saat ini)	kosong	Diisi mulai Sprint 3 (checkout)
XENDIT_WEBHOOK_VERIFICATION_TOKEN	Tidak (untuk saat ini)	kosong	Wajib sebelum webhook Xendit mode live (NF-05)
CORS_ALLOWED_ORIGINS	Tidak	http://localhost:3000	Daftar origin dipisah koma; di production isi domain resmi saja

4. Lapisan Data

4.1 Koneksi (internal/database/database.go)

NewPostgresPool membuat pool dengan MaxConns=20 , MinConns=2 , dan langsung melakukan Ping saat startup (fail-fast). NewRedisClient serupa untuk Redis, disiapkan untuk cache halaman publik dan job queue ringan (belum diisi).

4.2 Skema Database (migrasi 000001_init_schema)

9 entitas, persis mengikuti Bagian 4 Technical Design Document:

TABEL	FIELD KUNCI	CATATAN
users	id (UUID), email, password_hash, username, role, kyc_status	role: creator/admin
pages	id, user_id, theme, bio, avatar_url, is_published	1:1 dengan users
links	id, page_id, title, url, position, is_active	Urutan tampil via position
products	id, user_id, name, price_idr, file_key, cover_image_url, is_active	file_key tidak pernah diserialisasi ke JSON
orders	id, product_id, buyer_email, amount_idr, platform_fee_idr, status, psp_reference	status: pending/paid/failed/expired
payments	id, order_id, psp, method, psp_transaction_id, status, raw_webhook_payload	Menyimpan payload webhook mentah untuk audit
ledger_entries	id, user_id, order_id, type, amount_idr, balance_after	Append-only — tidak pernah di-UPDATE/DELETE, saldo direkonstruksi dari histori

payouts	id, user_id, amount_idr, destination_account, psp_disbursement_id, status	status: requested/processing/success/failed
analytics_events	id, page_id, event_type, link_id, referrer	event_type: click/view; kandidat partisi per bulan

Indeks: `links(page_id)` , `products(user_id)` , `orders(product_id)` , `ledger_entries(user_id)` ,
`analytics_events(page_id, created_at)` .

5. Middleware (`internal/middleware/middleware.go`)

MIDDLEWARE	FUNGSI
<code>CORS(allowedOrigins)</code>	Membatasi origin yang boleh memanggil API (dipisah koma). Tidak boleh wildcard <code>*</code> karena <code>AllowCredentials: true</code> .
<code>RequestLogger()</code>	Mencatat method, path, status, durasi tiap request — dasar observability sebelum APM lengkap dipasang.
<code>AuthRequired.jwtSecret)</code>	Memvalidasi header <code>Authorization: Bearer <token></code> , memverifikasi signing method HMAC, lalu menyisipkan <code>userID</code> (klaim <code>sub</code>) ke context Gin untuk dipakai handler berikutnya.

6. Routing Table (`internal/routes/routes.go`)

METHOD	PATH	AUTH	HANDLER	REF
GET	<code>/api/health</code>	—	<code>HealthHandler.Check</code>	Dipakai pipeline deploy
POST	<code>/api/v1/auth/register</code>	—	<code>AuthHandler.Register</code>	REQ-F-101, 102
POST	<code>/api/v1/auth/login</code>	—	<code>AuthHandler.Login</code>	—
GET	<code>/api/v1/pages/:username</code>	—	<code>PageHandler.GetPublicPage</code>	REQ-F-201
GET	<code>/api/v1/dashboard/products</code>	JWT	<code>ProductHandler.List</code>	—
POST	<code>/api/v1/dashboard/products</code>	JWT	<code>ProductHandler.Create</code>	REQ-F-301

Ditandai TODO langsung di `routes.go` : endpoint CRUD tautan (REQ-F-202/203), saldo & penarikan (REQ-F-501..504), analitik (REQ-F-601..603), checkout publik (REQ-F-401), dan webhook PSP (REQ-F-403 — **wajib** verifikasi signature sebelum diproses).

7. Detail Handler per Modul

7.1 Health (`handlers/health.go`)

Menguji koneksi database **dan** Redis secara aktif (bukan sekadar "server nyala") lewat `Ping` dengan timeout 3 detik. Mengembalikan HTTP 503 + `status: "degraded"` bila salah satu down. Inilah yang membuat health check di

pipeline deploy benar-benar berarti.

7.2 Auth (`handlers/auth.go`)

`Register` : validasi email/password (min 8 karakter)/username (3–30 karakter), hash password dengan `bcrypt.DefaultCost` , insert ke `users` dengan `role='creator'` , `kyc_status='unverified'` . `Login` : verifikasi `bcrypt.CompareHashAndPassword` , lalu menerbitkan JWT HS256 dengan klaim `sub` (user id), `exp` (24 jam), `iat` .

TODO tercatat di kode: membedakan error "email/username sudah dipakai" (unique constraint) dari error database lain agar pesan ke pengguna lebih jelas. OAuth Google (REQ-F-101), verifikasi email (REQ-F-104), KYC (REQ-F-105), dan revoke sesi (REQ-F-106) belum diimplementasikan.

7.3 Page (`handlers/page.go`)

`GetPublicPage` — implementasi REQ-F-201. Query `users+pages` (hanya yang `is_published=true`), lalu tautan aktif (urut `position`) dan produk aktif milik user tersebut. Mengembalikan 404 jika halaman tidak ada/belum publish. TODO tercatat: cek cache Redis dulu sebelum query database (key `"page:"+username`) — endpoint ini kandidat utama caching karena titik trafik tertinggi (NF-01, NF-02).

7.4 Product (`handlers/product.go`)

`Create` — REQ-F-301, dilindungi JWT. Produk dibuat dengan `is_active=false` secara default; TODO: baru boleh `is_active=true` setelah file diunggah lewat mekanisme terpisah (multipart/presigned upload — REQ-F-302/303/304, belum diimplementasikan). `List` mengembalikan seluruh produk milik kreator yang sedang login.

8. Migrasi Database

Ditangani lewat subcommand bawaan binary sendiri, `internal/migrate/migrate.go` (membungkus `golang-migrate/v4`) — **tidak perlu** instal CLI `golang-migrate` terpisah baik di lokal, CI, maupun VPS:

```
./api migrate up      # menerapkan seluruh migrasi
./api migrate down    # rollback satu langkah
./api migrate status  # cek versi migrasi saat ini (dan status "dirty")

# lewat Makefile:
make migrate-up
make migrate-down
make migrate-status
```

Subcommand ini dipakai juga oleh pipeline CI (sebelum `go test`) dan kedua workflow deploy (sebelum menyalakan versi image baru) — lihat Dokumentasi CI/CD & Deployment.

9. Development & Build

```
# Development harian (tanpa Docker)
cd apps/api
go mod tidy
go run main.go           # http://localhost:8080

# Build & verifikasi
go build -o bin/api main.go
go vet ./...
go test ./... -v          # butuh DATABASE_URL & REDIS_URL aktif untuk test integrasi

# Lewat Makefile
make run / make build / make test
```

10. Status Implementasi vs SRS

MODUL	STATUS	REF SRS
Autentikasi & Akun (register, login, JWT)	Sebagian	REQ-F-1xx
Halaman Publik & Tautan (baca)	Sebagian	REQ-F-2xx
Produk Digital & Katalog (buat, daftar)	Sebagian	REQ-F-3xx
Checkout & Pembayaran	Belum	REQ-F-4xx
Saldo & Penarikan Dana	Belum (skema DB sudah ada)	REQ-F-5xx
Analitik	Belum (skema DB sudah ada)	REQ-F-6xx
Panel Admin	Belum	REQ-F-7xx
Rate limiting API	Belum	NF-05
Background job/queue (worker)	Belum — belum ada subcommand <code>worker</code>	—